

به نام خدا

عنوان پروژه:

پیاده‌سازی FSM کنترلی برای مدیریت بلاک BRAM

استاد راهنما:

دکتر سلیمی

تهیه و تنظیم:

یاسین سموعی

عنوان درس:

کد نویسی VHDL

واحد دانشگاهی:

دانشگاه آزاد اسلامی واحد اصفهان

تاریخ ارائه:

آذر ماه ۱۴۰۴



فهرست مطالب

۴	مقدمه :
۵	FSM چیست ؟
۷	توضیح خط به خط ماژول ها.
۷	ماژول اول:(بلاک BRAM)
۸	توضیحات:
۱۴	شماتیک :
۱۵	ماژول دوم:(FSM)
۱۸	توضیحات :
۳۰	شماتیک :
۳۱	ماژول سوم:(Top Module)
۳۴	توضیحات :
۴۰	شماتیک :
۴۱	ماژول چهارم:(Test Bench)
۴۷	توضیحات :
۵۴	بررسی نتایج شبیه سازی:
۵۵	تحلیل عملکرد سیستم
۵۵	مشخصات معماری سیستم
۵۵	تاخیرهای زمانی (Latency) - تحلیل دقیق
۵۵	الف) عملیات نوشتن (Write)
۵۶	ب) عملیات خواندن (Read)

- ۵۷..... دلیل تاخیرهای مشاهده شده
- ۵۷..... تاخیر نوشتن (۴ سیکل):
- ۵۸..... تاخیر خواندن (۵ سیکل):
- ۵۸..... صحت عملکرد
- ۵۹..... تاخیرهای عملیاتی
- ۵۹..... دیگرام حالت‌های FSM
- ۵۹..... پیشنهادات بهینه‌سازی
- ۶۰..... نتیجه‌گیری نهایی
- ۶۳..... مقایسه معماری‌های حافظه Single-Port ، Dual-Port و FIFO:

مقدمه :

این پروژه با هدف پیاده‌سازی یک ماشین حالت متناهی (FSM) برای کنترل بلاک حافظه BRAM طراحی و اجرا شده است. ساختار کلی پروژه در قالب سه فایل مجزا سازمان‌دهی شده که هر کدام وظیفه مشخصی را بر عهده دارند :

ماژول ۱ : bram_sram

این بخش به عنوان حافظه‌ی پایه (SRAM/BRAM) عمل می‌کند. وظیفه‌ی آن ذخیره‌سازی و بازیابی داده‌ها بر اساس آدرس ورودی است. سیگنال‌های ورودی شامل ساعت، ریست، آدرس، داده‌ی ورودی و سیگنال نوشتن هستند و خروجی داده‌ی خوانده‌شده از حافظه می‌باشد.

ماژول ۲ : bram_controller

این بخش مسئولیت کنترل عملیات خواندن و نوشتن در حافظه BRAM را بر اساس سیگنال‌های کنترلی FSM بر عهده دارد. وظایف اصلی آن شامل مدیریت آدرس‌دهی، فعال‌سازی سیگنال‌های نوشتن و خواندن، و هماهنگی داده‌های ورودی و خروجی است.

ماژول ۳ : top_bram_fsm

این ماژول نقش سطح بالای طراحی را ایفا می‌کند و ترکیبی از ماژول حافظه و ماژول کنترلی است. در واقع، این بخش به عنوان مدار اصلی، ارتباط بین FSM و حافظه BRAM را برقرار کرده و کل سیستم را یکپارچه می‌سازد.

ماژول ۴ : tb_top_bram_fsm

این فایل به عنوان **testbench** طراحی شده و وظیفه شبیه‌سازی و تست عملکرد ماژول اصلی را دارد. با استفاده از این بخش، صحت عملکرد FSM و حافظه بررسی شده و نتایج شبیه‌سازی برای تحلیل و ارزیابی پروژه مورد استفاده قرار می‌گیرد.

در ادامه، هر یک از این ماژول‌ها به صورت جداگانه مورد بررسی قرار خواهند گرفت تا ساختار داخلی، سیگنال‌ها و وظایف آن‌ها به طور کامل توضیح داده شود.

FSM چیست؟

FSM یعنی چی؟

FSM یا **Finite State Machine** یک مدل کنترلی است که می‌گوید: سیستم در هر لحظه فقط می‌تواند در یک "حالت" باشد، و بر اساس ورودی‌ها از یک حالت به حالت دیگری تغییر می‌کند. یعنی مثل یک جدول قوانین:

- اگر در حالت A هستی و ورودی X بیاد $\Rightarrow$ برو به حالت B
- اگر در حالت B هستی و ورودی Y بیاد $\Rightarrow$ برو به حالت C

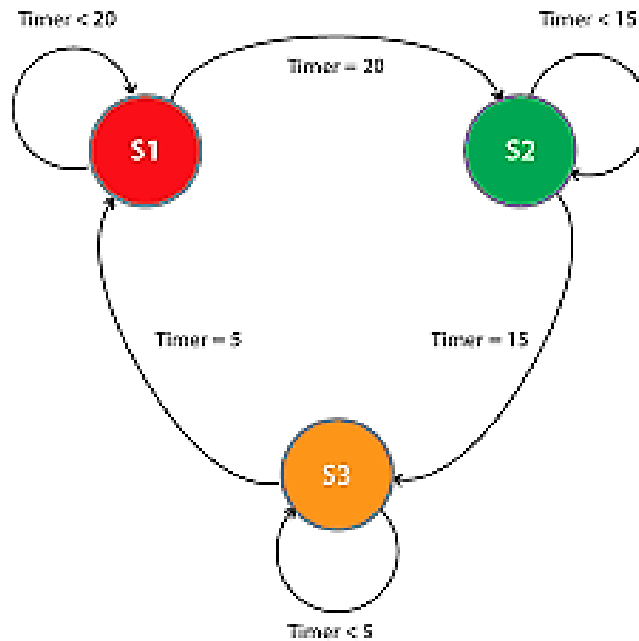
مثال واقعی: چراغ راهنمایی

فرض کنید یک چراغ راهنمایی سه رنگ داریم: قرمز، سبز، زرد.

- حالت‌ها: قرمز، سبز، زرد
- ورودی: زمان یا تایمر
- انتقال حالت‌ها:

- وقتی تایمر قرمز تمام شد $\rightarrow$ برو به سبز
- وقتی تایمر سبز تمام شد $\rightarrow$ برو به زرد
- وقتی تایمر زرد تمام شد $\rightarrow$ برو به قرمز

این دقیقاً یک FSM هست. چراغ راهنمایی هیچ وقت همزمان در دو حالت نیست، همیشه یکی از سه حالت رو داره.



مثال مرتبط با پروژه: کنترل حافظه BRAM

حالا همین مفهوم را داخل پروژه بررسی کنیم :

- حالت‌ها : (IDLE) بیکار، (WRITE) نوشتن، (READ) خواندن
- ورودی‌ها: سیگنال‌های we نوشتن/خواندن، clk ساعت، rst ریست
- انتقال حالت‌ها :
- اگر ریست فعال باشد ← برو به حالت IDLE
- اگر $we=1$ باشد ← برو به حالت WRITE و داده رو داخل حافظه ذخیره کن
- اگر $we=0$ باشد ← برو به حالت READ و داده رو از حافظه بخون

جمع‌بندی

- FSM مثل یک مدیر ترافیک برای مدار عمل می‌کند .
- حافظه‌ی BRAM خودش فقط داده ذخیره می‌کند، ولی FSM تصمیم می‌گیرد چه زمانی بخوانیم، چه زمانی بنویسیم، و چه زمانی هیچ کاری نکنیم .
- این باعث می‌شود مدار قابل پیش‌بینی، منظم و بدون تداخل کار کند .

توضیح خط به خط ماژول ها

ماژول اول:

کد:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bram_sram is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk : in std_logic;
    rst : in std_logic;
    we : in std_logic; -- '1' => write
    addr : in std_logic_vector(ADDR_WIDTH-1 downto 0);
    din : in std_logic_vector(DATA_WIDTH-1 downto 0);
    dout : out std_logic_vector(DATA_WIDTH-1 downto 0)
  );
end entity;

architecture rtl of bram_sram is
  type ram_t is array (0 to (2**ADDR_WIDTH)-1) of
    std_logic_vector(DATA_WIDTH-1 downto 0);
  signal mem : ram_t := (others => (others => '0'));
  signal addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
  signal dout_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
begin
  process(clk)
```

```

begin
  if rising_edge(clk) then
    if rst = '1' then
      -- Initialize all memory to zero
      for i in 0 to (2**ADDR_WIDTH)-1 loop
        mem(i) <= (others => '0');
      end loop;
      addr_reg <= (others => '0');
      dout_reg <= (others => '0');
    else
      -- Register address for read (1 cycle latency)
      addr_reg <= addr;

      -- Write operation
      if we = '1' then
        mem(to_integer(unsigned(addr))) <= din;
      end if;

      -- Read operation (with 1 cycle latency)
      dout_reg <= mem(to_integer(unsigned(addr_reg)));
    end if;
  end if;
end process;

dout <= dout_reg;

end architecture;

```

توضیحات:

مرور کلی

این ماژول یک حافظه Single-Port BRAM/SRAM همزمان (synchronous) با تأخیر یک سیکل برای خواندن است. آدرس خواندن در رجیستر ذخیره می‌شود و داده خروجی در سیکل بعدی آماده می‌گردد؛ نوشتن نیز همزمان با لبه بالارونده کلاک انجام می‌شود. ریست همزمان است و علاوه بر پاک‌سازی رجیسترها، کل حافظه را صفر می‌کند.

توضیح خطبه خط

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity bram_sram is
    generic(
        ADDR_WIDTH : integer := 8;
        DATA_WIDTH : integer := 8
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        we : in std_logic; -- '1' => write
        addr : in std_logic_vector(ADDR_WIDTH-1 downto 0);
        din : in std_logic_vector(DATA_WIDTH-1 downto 0);
        dout : out std_logic_vector(DATA_WIDTH-1 downto 0)
    );
end entity;
```

• ژنریک ها :

◦ ADDR_WIDTH : عرض آدرس؛ اندازه حافظه برابر $(2^{\text{ADDR_WIDTH}})$ کلمه است.

◦ DATA_WIDTH: عرض داده؛ تعداد بیت هر کلمه.

• پورت‌ها :

- Clk: کلاک هم‌زمان برای عملیات خواندن/نوشتن.
- Rst: ریست هم‌زمان؛ وقتی ۱ شود، حافظه و رجیسترها صفر می‌شوند.
- We: سیگنال نوشتن؛ وقتی ۱ باشد، در همان سیکل روی آدرس جاری می‌نویسد.
- Addr: آدرس ورودی برای خواندن/نوشتن.
- Din: داده ورودی برای نوشتن.
- Dout: داده خروجی خوانده‌شده (با یک سیکل تأخیر).

architecture rtl of bram_sram is

```
type ram_t is array (0 to (2**ADDR_WIDTH)-1) of
std_logic_vector(DATA_WIDTH-1 downto 0);

signal mem : ram_t := (others => (others => '0'));

signal addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) :=
(others => '0');

signal dout_reg : std_logic_vector(DATA_WIDTH-1 downto 0) :=
(others => '0');
```

• نوع حافظه :

- ram_t: آرایه‌ای با اندازه $(2^{\{ADDR_WIDTH\}})$ از کلمات DATA_WIDTH بیتی.

• سیگنال حافظه :

- Mem: نمونه‌ی آرایه حافظه؛ با صفر مقداردهی اولیه شده (برای شبیه‌سازی).

• رجیستر آدرس :

- addr_reg: آدرس خواندن ثبت‌شده برای اعمال تأخیر یک سیکل.

- رجیستر خروجی :

- `dout_reg`: خروجی ثبت شده؛ `dout` از این رجیستر تغذیه می شود.

```
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        -- Initialize all memory to zero
        for i in 0 to (2**ADDR_WIDTH)-1 loop
          mem(i) <= (others => '0');
        end loop;
        addr_reg <= (others => '0');
        dout_reg <= (others => '0');
```

- پردازنده همزمان با کلاک :

- همه چیز روی لبه بالارونده رخ می دهد.

- ریست همزمان :

- حلقه `for` کل حافظه را صفر می کند.

- `addr_reg` و `dout_reg` صفر می شوند.

```
else
  -- Register address for read (1 cycle
  latency)
  addr_reg <= addr;
```

```

-- Write operation
if we = '1' then
    mem(to_integer(unsigned(addr))) <= din;
end if;

-- Read operation (with 1 cycle latency)
dout_reg <=
mem(to_integer(unsigned(addr_reg)));
end if;
end if;
end process;

```

• ثبت آدرس خواندن :

◦ `addr_reg <= addr;` آدرس فعلی را ذخیره می کند تا در سیکل بعدی برای خواندن استفاده شود (تأخیر ۱ سیکل).

• نوشتن همزمان :

◦ اگر `we='1'`، در همان سیکل روی `mem` با آدرس فعلی `addr` می نویسد؛ تبدیل آدرس با `unsigned` و سپس `to_integer` انجام می شود.

• خواندن با تأخیر :

◦ `dout_reg <= mem(... addr_reg ...)` داده را از آدرسی که سیکل قبل ثبت شده بود می خواند. بنابراین خروجی متناظر با آدرسی است که یک سیکل قبل روی `addr` اعمال شده بود.

```
dout <= dout_reg;  
end architecture;
```

- خروجی رجیسترشده :

- پورت dout از رجیستر dout_reg تغذیه می‌شود؛ خواندن کاملاً رجیسترشده و پایدار است.

رفتار و تایمینگ

تأخیر خواندن یک سیکل

- ورودی آدرس :

- در سیکل (n) : addr نمونه‌برداری و در addr_reg ذخیره می‌شود.

- خروجی داده :

- در سیکل (n+1) : dout_reg از mem(addr_reg) لود می‌شود و dout همان را نشان می‌دهد.

- نتیجه :

- هر آدرسی که اعمال می‌کنیم، داده‌اش یک سیکل بعد روی خروجی ظاهر می‌شود.

نوشتن و خواندن هم‌زمان

- نوشتن :

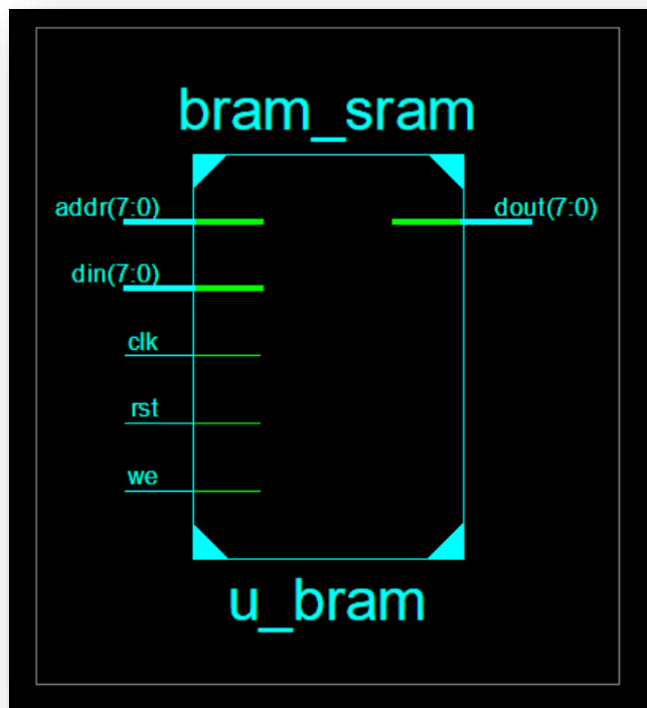
- اگر we='1' در سیکل (n) ، mem(addr) با din به‌روز می‌شود.

- خواندن همان سیکل :

- خواندن از `mem(addr_reg)` انجام می‌شود؛ یعنی آدرس سیکل قبل. اگر در همان سیکل روی همان آدرس فعلاً در `addr` بنویسید، خروجی آن سیکل تحت تأثیر نیست و در سیکل بعد داده جدید دیده می‌شود.

حالات مرزی و سیاست `read-during-write`

- نوشتن و خواندن به یک آدرس واحد :
 - چون خواندن از `addr_reg` است، اگر در سیکل (n) روی `addr=A` بنویسید، `dout` در سیکل (n) هنوز داده‌ی آدرس سیکل قبل را نشان می‌دهد. در سیکل $(n+1)$ ، `addr_reg=A` شده و `dout` مقدار جدید را نشان خواهد داد.
- `read-during-write` به همان آدرس در یک سیکل :
 - این طراحی به شکل «`write-first`» یا «`read-first`» در همان سیکل تعریف نمی‌کند، چون خواندن از آدرس ثبت‌شده‌ی سیکل قبل انجام می‌شود. عملاً بازه‌ی یک سیکل تأخیر خواندن، `ambiguity` رفتار را حذف کرده و خروجی همان سیکل تحت نوشتن جاری قرار نمی‌گیرد.
- تبدیل نوع آدرس :
 - `to_integer(unsigned(addr))` استفاده‌ی امن از `numeric_std` است؛ از `std_logic_unsigned` یا تبدیل‌های ناسازگار اجتناب شده است.



ماژول دوم:

کد:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bram_controller is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk      : in  std_logic;
    rst      : in  std_logic;
    cmd      : in  std_logic_vector(1 downto 0);
```

```

    cmd_valid : in  std_logic;
    addr_in   : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in   : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_dout : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_we   : out std_logic;
    bram_addr : out std_logic_vector(ADDR_WIDTH-1 downto 0);
    bram_din  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid: out std_logic;
    busy      : out std_logic
  );
end entity;

architecture rtl of bram_controller is
  type state_t is (IDLE, READ_WAIT1, READ_WAIT2, READ_DONE, WRITE_DO,
WRITE_DONE);
  signal state      : state_t := IDLE;
  signal addr_reg   : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others
=> '0');
  signal data_reg   : std_logic_vector(DATA_WIDTH-1 downto 0) := (others
=> '0');
  signal data_out_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others
=> '0');
  signal bram_we_reg  : std_logic := '0';
  signal bram_addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) :=
(others => '0');
  signal bram_din_reg  : std_logic_vector(DATA_WIDTH-1 downto 0) :=
(others => '0');

begin
  -- Registered outputs
  bram_we    <= bram_we_reg;
  bram_addr <= bram_addr_reg;
  bram_din  <= bram_din_reg;
  data_out  <= data_out_reg;

  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        -- Reset all registers
        state      <= IDLE;
        addr_reg   <= (others => '0');

```



```

data_reg    <= (others => '0');
data_out_reg <= (others => '0');
bram_we_reg <= '0';
bram_addr_reg <= (others => '0');
bram_din_reg <= (others => '0');
data_valid  <= '0';
busy        <= '0';
else
    -- Default values
    bram_we_reg <= '0';
    data_valid  <= '0';

    case state is
        when IDLE =>
            busy <= '0';

            if cmd_valid = '1' then
                busy <= '1';
                addr_reg <= addr_in;
                data_reg <= data_in;

                case cmd is
                    when "01" => -- Read command
                        -- Apply address to BRAM immediately
                        bram_addr_reg <= addr_in;
                        state <= READ_WAIT1;

                    when "10" => -- Write command
                        -- Apply address and data to BRAM immediately
                        bram_addr_reg <= addr_in;
                        bram_din_reg  <= data_in;
                        bram_we_reg    <= '1'; -- Activate write enable
                        state <= WRITE_D0;

                    when others => -- Invalid command
                        state <= IDLE;
                        busy <= '0';
                end case;
            end if;

        when READ_WAIT1 =>
            -- First wait state for BRAM read latency
            bram_addr_reg <= addr_reg;

```

```

        state <= READ_WAIT2;

    when READ_WAIT2 =>
        -- Second wait state (BRAM has 1 cycle latency + we need one
more)
        bram_addr_reg <= addr_reg;
        state <= READ_DONE;

    when READ_DONE =>
        -- Capture data from BRAM
        data_out_reg <= bram_dout;
        data_valid <= '1';
        state <= IDLE;

    when WRITE_DO =>
        -- Write was performed, keep we active for one cycle
        bram_addr_reg <= addr_reg;
        bram_din_reg <= data_reg;
        bram_we_reg <= '1';
        state <= WRITE_DONE;

    when WRITE_DONE =>
        -- Finish write operation
        bram_we_reg <= '0';
        state <= IDLE;

    end case;
end if;
end if;
end process;

end architecture;

```

توضیحات :

این کد یک کنترلر **FSM** برای ماژول حافظه **BRAM** است. وظیفه‌اش این است که دستورات خواندن و نوشتن را مدیریت کند، سیگنال‌های لازم را به **BRAM** بدهد، و داده‌ها را با تأخیر درست تحویل دهد.

◆ تعريف Entity

```
entity bram_controller is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk, rst      : in  std_logic;
    cmd           : in  std_logic_vector(1 downto
0);
    cmd_valid     : in  std_logic;
    addr_in       : in
std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in       : in
std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_dout     : in
std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_we       : out std_logic;
    bram_addr     : out
std_logic_vector(ADDR_WIDTH-1 downto 0);
    bram_din      : out
std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out      : out
std_logic_vector(DATA_WIDTH-1 downto 0);
```

```

    data_valid      : out std_logic;
    busy            : out std_logic
);
end entity;

```

- **cmd**: دستور (۲ بیت) "01" خواندن، "10" نوشتن.
- **cmd_valid**: معتبر بودن دستور.
- **addr_in**: آدرس حافظه.
- **data_in**: داده برای نوشتن.
- **bram_dout**: داده خروجی از BRAM.
- **bram_we, bram_addr, bram_din**: سیگنال‌های کنترل و داده برای BRAM.
- **data_out, data_valid**: خروجی داده خوانده‌شده و پرچم معتبر بودن آن.
- **Busy**: نشان می‌دهد کنترلر مشغول اجرای دستور است.

◆ تعریف State Machine

```

type state_t is (IDLE, READ_WAIT1, READ_WAIT2,
READ_DONE, WRITE_DO, WRITE_DONE);
signal state : state_t := IDLE;

```

- **IDLE**: حالت بیکار، منتظر دستور.
- **READ_WAIT1 / READ_WAIT2**: حالت‌های انتظار برای خواندن چون BRAM یک سیکل تأخیر دارد.

- **READ_DONE**: داده خوانده شده و آماده تحویل است.

- **WRITE_DO**: اجرای نوشتن.

- **WRITE_DONE**: پایان نوشتن.

♦ رفتار خط به خط در هر کلاک

۱. ریست (**rst='1'**)

- همه رجیسترها صفر می‌شوند.

- FSM به حالت IDLE برمی‌گردد.

- سیگنال‌های خروجی (**busy, data_valid, bram_we**) غیر فعال می‌شوند.

۲. حالت IDLE

- **busy='0'** کنترلر آزاد است.

- اگر **cmd_valid='1'** شود :

- **busy='1'** کنترلر مشغول می‌شود.

- آدرس و داده ورودی در رجیستر ذخیره می‌شوند.

- اگر دستور "01" باشد ← شروع خواندن: آدرس به BRAM داده می‌شود و

- می‌رویم به **READ_WAIT1**.

- اگر دستور "10" باشد ← شروع نوشتن: آدرس و داده به BRAM داده

- می‌شوند، **we='1'** فعال می‌شود و می‌رویم به **WRITE_DO**.

۳. خواندن (READ_WAIT1 → READ_WAIT2 → READ_DONE)

- **READ_WAIT1**: آدرس دوباره روی BRAM اعمال می‌شود.
- **READ_WAIT2**: یک سیکل دیگر صبر می‌کنیم چون BRAM یک سیکل تأخیر دارد.
- **READ_DONE**: داده از bram_dout گرفته می‌شود، در data_out_reg ذخیره می‌شود، data_valid='1' می‌شود. سپس FSM به IDLE برمی‌گردد.
- 🕒 **تأخیر خواندن**: از لحظه‌ای که دستور خواندن داده می‌شود تا زمانی که داده معتبر روی خروجی بیاید، حدود ۲ سیکل کلاک طول می‌کشد.

۴. نوشتن (WRITE_DO → WRITE_DONE)

- **WRITE_DO**: داده و آدرس روی BRAM اعمال می‌شوند، we='1' فعال است.
- **WRITE_DONE**: سیگنال we غیر فعال می‌شود و FSM به IDLE برمی‌گردد.
- 🕒 **تأخیر نوشتن**: داده همان سیکل نوشته می‌شود، و در سیکل بعد عملیات پایان می‌یابد.

♦ جمع‌بندی رفتار

- **خواندن**: دستور ← "01"؛ داده بعد از ۲ سیکل انتظار روی خروجی ظاهر می‌شود.

- نوشتن :دستور ← "10": داده همان لحظه نوشته می‌شود، و بعد از ۱ سیکل عملیات تمام می‌شود.

- Busy: نشان می‌دهد کنترلر در حال اجرای دستور است.

- data_valid: فقط در لحظه‌ای که داده خوانده شده معتبر است.

توضیح خطبه‌خط:

در ادامه، همه چیز را از بخش architecture تا انتهای کد، خطبه‌خط و با تحلیل رفتاری روی کلاک توضیح داده شده.

معماری و سیگنال‌ها

```
signal state      : state_t := IDLE;
```

- حالت فعلی : رجیستر حالت که با IDLE شروع می‌شود و در هر لبه بالارونده کلاک به‌روزرسانی می‌شود.

```
signal addr_reg    : std_logic_vector(ADDR_WIDTH-1  
downto 0) := (others => '0');
```

- ثبت آدرس ورودی : آدرس فرمان دریافتی را نگه می‌دارد تا در طول عملیات پایدار باشد.

```
signal data_reg    : std_logic_vector(DATA_WIDTH-1  
downto 0) := (others => '0');
```

- ثبت داده ورودی : داده نوشتن را نگه می‌دارد تا هنگام ارسال به BRAM پایدار باشد.

```
signal data_out_reg :  
std_logic_vector(DATA_WIDTH-1 downto 0) := (others  
=> '0');
```

- رجیستر خروجی داده : داده خوانده شده از BRAM را ذخیره می کند تا از طریق پورت data_out ارائه شود.

```
signal bram_we_reg : std_logic := '0';
```

- رجیستر سیگنال نوشتن BRAM : فعال سازی نوشتن روی BRAM را با زمان بندی مشخص کنترل می کند.

```
signal bram_addr_reg :  
std_logic_vector(ADDR_WIDTH-1 downto 0) := (others  
=> '0');
```

- رجیستر آدرس BRAM : آدرسی که به خود BRAM اعمال می شود، رجیستر شده است تا با کلاک همزمان باشد.

```
signal bram_din_reg :  
std_logic_vector(DATA_WIDTH-1 downto 0) := (others  
=> '0');
```

- رجیستر داده ورودی BRAM : داده ای که باید نوشته شود را رجیستر می کند تا در لبه کلاک معتبر باشد.

نگاشت خروجی های رجیستر شده

```
begin  
-- Registered outputs  
bram_we    <= bram_we_reg;  
bram_addr <= bram_addr_reg;
```



```
bram_din  <= bram_din_reg;
```

```
data_out  <= data_out_reg;
```

- خروجی‌های پایدار : پورت‌های خروجی مستقیماً از رجیسترها تغذیه می‌شوند. این کار زمان‌بندی را بهبود می‌دهد و نویز ترکیبی را حذف می‌کند.

فرآیند اصلی هم‌زمان با کلاک

```
process(clk)
```

```
begin
```

```
    if rising_edge(clk) then
```

- لبه بالارونده کلاک : تمامی تغییرات حالت و رجیسترها در این لحظه اتفاق می‌افتد؛ طراحی کاملاً هم‌زمان است.

```
        if rst = '1' then
```

```
            -- Reset all registers
```

```
            state      <= IDLE;
```

```
            addr_reg   <= (others => '0');
```

```
            data_reg    <= (others => '0');
```

```
            data_out_reg <= (others => '0');
```

```
            bram_we_reg <= '0';
```

```
            bram_addr_reg <= (others => '0');
```

```
            bram_din_reg <= (others => '0');
```

```
            data_valid  <= '0';
```

```
            busy        <= '0';
```

- ریست همزمان : با فعال بودن rst، تمامی رجیسترها صفر می‌شوند، FSM به IDLE می‌رود، سیگنال‌های وضعیت (busy, data_valid) غیرفعال می‌شوند، و هیچ عملیاتی روی BRAM انجام نمی‌شود.

```
else
```

```
-- Default values
```

```
bram_we_reg <= '0';
```

```
data_valid <= '0';
```

- مقادیر پیش‌فرض هر سیکل : قبل از بررسی حالت‌ها، نوشتن BRAM غیرفعال و data_valid صفر می‌شود. سپس حالت جاری می‌تواند این مقادیر را در همان سیکل دوباره فعال کند (در حالت‌های خاص).

```
case state is
```

```
when IDLE =>
```

```
busy <= '0';
```

- حالت IDLE : کنترلر آزاد است و تراکنشی در جریان نیست.

```
if cmd_valid = '1' then
```

```
busy <= '1';
```

```
addr_reg <= addr_in;
```

```
data_reg <= data_in;
```

- دریافت فرمان : با معتبر شدن فرمان، کنترلر مشغول می‌شود و آدرس/داده ورودی را در رجیسترها ذخیره می‌کند تا طی تراکنش ثابت بمانند.

```
case cmd is
```

```
when "01" => -- Read command
```

```
-- Apply address to BRAM immediately
```

```
bram_addr_reg <= addr_in;
```

```
state <= READ_WAIT1;
```

- **شروع خواندن** : برای خواندن، آدرس فوراً روی BRAM اعمال می‌شود و به حالت انتظار اول می‌رویم. چون BRAM خواندن رجیسترشده دارد، خروجی داده با تأخیر ظاهر می‌شود.

```
when "10" => -- Write command
```

```
-- Apply address and data to BRAM immediately
```

```
bram_addr_reg <= addr_in;
```

```
bram_din_reg <= data_in;
```

```
bram_we_reg <= '1'; -- Activate write enable
```

```
state <= WRITE_DO;
```

- **شروع نوشتن** : آدرس و داده فوراً روی BRAM اعمال می‌شوند و we فعال می‌شود. وارد مرحله اجرای نوشتن می‌شویم.

```
when others => -- Invalid command
```

```
state <= IDLE;
```

```
busy <= '0';
```

- **فرمان نامعتبر** : کنترلر به حالت بیکار برمی‌گردد و مشغولیت را لغو می‌کند.

```
end if;
```

- **پایان بررسی فرمان در IDLE** .

```
when READ_WAIT1 =>
```

```
-- First wait state for BRAM read latency
```

```
bram_addr_reg <= addr_reg;
```

```
state <= READ_WAIT2;
```

- **انتظار خواندن ۱** : آدرس ثبت شده مجدداً روی BRAM نگه‌داری/اعمال می‌شود. این مرحله برای تطبیق با تأخیر داخلی خواندن BRAM است.

```
when READ_WAIT2 =>
```

```
-- Second wait state (BRAM has 1 cycle latency  
+ we need one more)
```

```
bram_addr_reg <= addr_reg;
```

```
state <= READ_DONE;
```

- **انتظار خواندن ۲** : یک سیکل دیگر آدرس را پایدار نگه می‌داریم تا خروجی BRAM تثبیت شود. سپس آماده گرفتن داده هستیم.

```
when READ_DONE =>
```

```
-- Capture data from BRAM
```

```
data_out_reg <= bram_dout;
```

```
data_valid <= '1';
```

```
state <= IDLE;
```

- **پایان خواندن** : داده خروجی BRAM نمونه‌برداری و در data_out_reg ذخیره می‌شود؛ data_valid برای اعلام آماده بودن داده یک سیکل فعال می‌گردد. سپس به IDLE برمی‌گردیم.

```
when WRITE_DO =>
```

```
-- Write was performed, keep we active for one  
cycle
```

```
bram_addr_reg <= addr_reg;
```

```
bram_din_reg <= data_reg;
```

```
bram_we_reg <= '1';
```

```
state <= WRITE_DONE;
```

- **اجرای نوشتن** : آدرس و داده رجیسترشده به BRAM اعمال می شود و we در این سیکل فعال می ماند تا نوشتن انجام شود. سپس برای پایان نوشتن آماده می شویم.

```
when WRITE_DONE =>
```

```
-- Finish write operation
```

```
bram_we_reg <= '0';
```

```
state <= IDLE;
```

- **پایان نوشتن** : سیگنال نوشتن غیرفعال می شود و کنترلر به حالت بیکار بازمی گردد. نوشتن از همان سیکل قبلی تثبیت شده است.

```
end case;
```

```
end if;
```

```
end if;
```

```
end process;
```

- **پایان فرآیند** : تمام حالات در هر لبه کلاک بررسی و اعمال شدند.

```
end architecture;
```

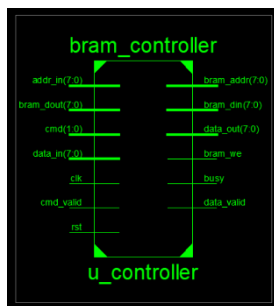
- **پایان معماری**.

نکات رفتاری و طراحی

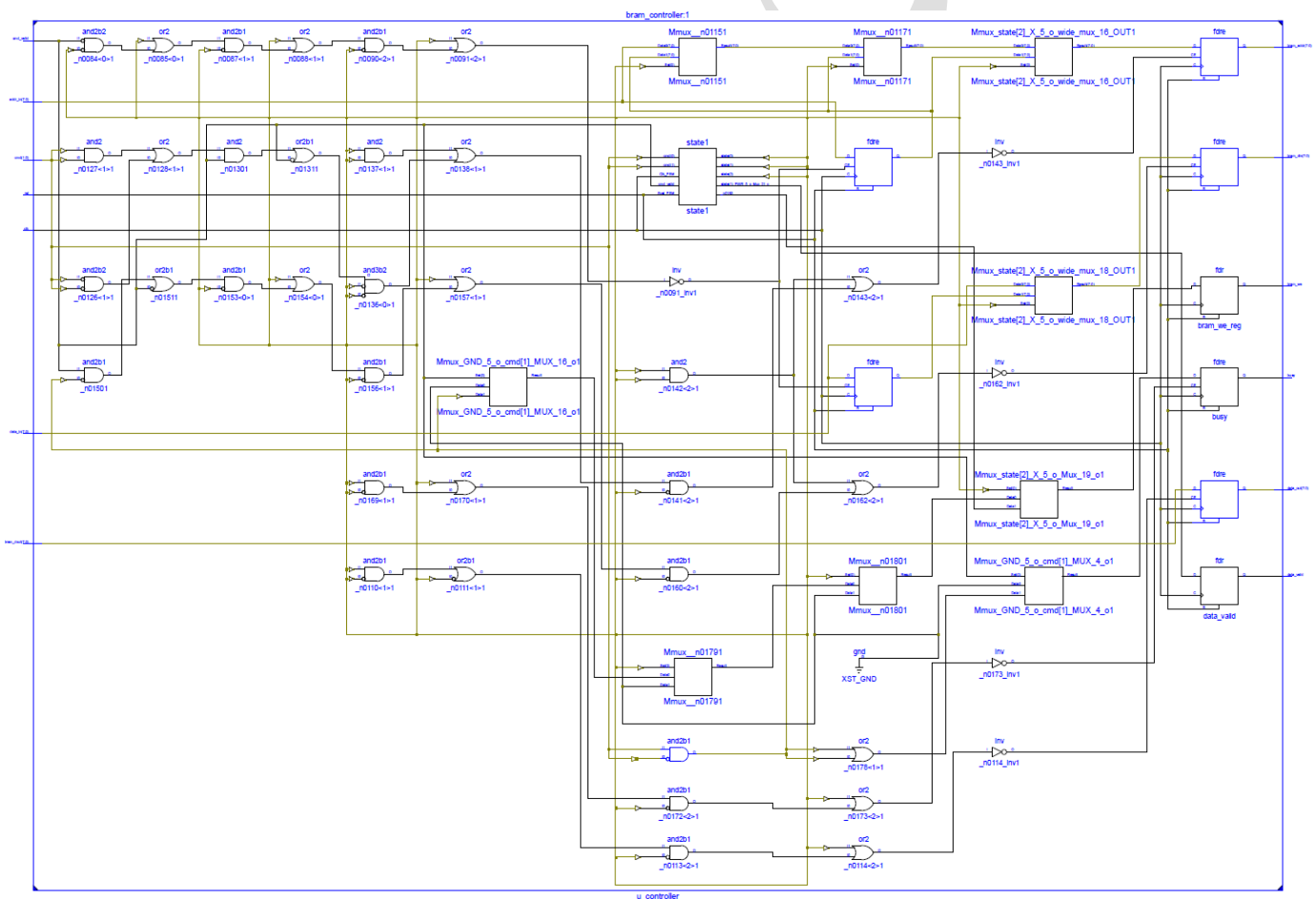
- **پایداری آدرس/داده** : استفاده از addr_reg و data_reg تضمین می کند که آدرس/داده در کل تراکنش ثابت و قابل اعتماد باشند.

- رجیستر کردن خروجی‌ها: `bram_addr`, `bram_din`, `bram_we`, `data_out` همگی رجیستر شده هستند؛ این کار مسیرهای زمانی را کوتاه و فرکانس قابل دستیابی را بالاتر می‌برد.

- چرایی دو مرحله انتظار خواندن : چون `BRAM` ما خواندن رجیستر شده با تأخیر یک سیکل دارد، کنترلر یک سیکل اضافی برای پایداری و همگام‌سازی خروجی در نظر گرفته تا در `READ_DONE` با اطمینان نمونه‌برداری کند. این الگو در سیستم‌های پایدار و قابل پیش‌بینی مفید است.



شماتیک :



ماژول سوم:

کد:

```
-- file: top_bram_fsm.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity top_bram_fsm is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk      : in  std_logic;
    rst      : in  std_logic;
    cmd      : in  std_logic_vector(1 downto 0);
    cmd_valid : in  std_logic;
    addr_in  : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in  : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid : out std_logic;
    busy     : out std_logic
  );
end entity;

architecture structural of top_bram_fsm is
  signal bram_we_s  : std_logic;
  signal bram_addr_s : std_logic_vector(ADDR_WIDTH-1 downto 0);
  signal bram_din_s  : std_logic_vector(DATA_WIDTH-1 downto 0);
  signal bram_dout_s : std_logic_vector(DATA_WIDTH-1 downto 0);

  -- Internal signals that can be probed in simulation
  signal controller_state_debug : std_logic_vector(2 downto 0);
  signal bram_we_debug          : std_logic;
  signal bram_addr_debug        : std_logic_vector(ADDR_WIDTH-1 downto 0);
  signal bram_din_debug         : std_logic_vector(DATA_WIDTH-1 downto 0);
  signal bram_dout_debug        : std_logic_vector(DATA_WIDTH-1 downto 0);
```

```

-- Component declarations
component bram_controller
  generic(
    ADDR_WIDTH : integer;
    DATA_WIDTH : integer
  );
  port(
    clk      : in  std_logic;
    rst      : in  std_logic;
    cmd      : in  std_logic_vector(1 downto 0);
    cmd_valid : in  std_logic;
    addr_in  : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in  : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_dout : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_we  : out std_logic;
    bram_addr : out std_logic_vector(ADDR_WIDTH-1 downto 0);
    bram_din  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid : out std_logic;
    busy      : out std_logic
  );
end component;

component bram_sram
  generic(
    ADDR_WIDTH : integer;
    DATA_WIDTH : integer
  );
  port(
    clk : in  std_logic;
    rst : in  std_logic;
    we  : in  std_logic;
    addr : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
    din  : in  std_logic_vector(DATA_WIDTH-1 downto 0);
    dout : out std_logic_vector(DATA_WIDTH-1 downto 0)
  );
end component;

begin
  -- Connect debug signals
  bram_we_debug  <= bram_we_s;
  bram_addr_debug <= bram_addr_s;

```



```

    bram_din_debug <= bram_din_s;
    bram_dout_debug <= bram_dout_s;

    -- Instantiate controller
    u_controller: bram_controller
        generic map (
            ADDR_WIDTH => ADDR_WIDTH,
            DATA_WIDTH => DATA_WIDTH
        )
        port map (
            clk      => clk,
            rst      => rst,
            cmd      => cmd,
            cmd_valid => cmd_valid,
            addr_in  => addr_in,
            data_in  => data_in,
            bram_dout => bram_dout_s,
            bram_we  => bram_we_s,
            bram_addr => bram_addr_s,
            bram_din  => bram_din_s,
            data_out  => data_out,
            data_valid => data_valid,
            busy      => busy
        );

    -- Instantiate BRAM
    u_bram: bram_sram
        generic map (
            ADDR_WIDTH => ADDR_WIDTH,
            DATA_WIDTH => DATA_WIDTH
        )
        port map (
            clk => clk,
            rst => rst,
            we  => bram_we_s,
            addr => bram_addr_s,
            din  => bram_din_s,
            dout => bram_dout_s
        );
end architecture;

```

توضیحات :

این ماژول با نام `top_bram_fsm` نقش ماژول سطح بالا را در طراحی ایفا می‌کند و وظیفه‌ی آن اتصال دو زیرماژول اصلی یعنی `bram_controller` (کنترلر حافظه) و `bram_sram` (ماژول حافظه) به یکدیگر است. در این سطح، ورودی‌های کاربر دریافت می‌شوند، کنترلر عملیات خواندن یا نوشتن را مدیریت می‌کند، و حافظه طبق فرمان کنترلر داده را ذخیره یا بازیابی می‌نماید. خروجی نهایی از حافظه گرفته شده و به کاربر ارائه می‌شود.

وجودیت (Entity)

```
entity top_bram_fsm is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk      : in  std_logic;
    rst      : in  std_logic;
    cmd      : in  std_logic_vector(1 downto 0);
    cmd_valid : in  std_logic;
    addr_in   : in  std_logic_vector(ADDR_WIDTH-1
downto 0);
    data_in   : in  std_logic_vector(DATA_WIDTH-1
downto 0);
    data_out  : out std_logic_vector(DATA_WIDTH-1
downto 0);
```

```
data_valid: out std_logic;
busy      : out std_logic
);
end entity;
```

🔑 توضیح پورت‌ها

• ورودی‌ها:

- Clk: کلاک سیستم.
- Rst: ریست هم‌زمان.
- Cmd: فرمان خواندن یا نوشتن (۲ بیت).
- cmd_valid: اعتبار فرمان.
- addr_in: آدرس حافظه.
- data_in: داده‌ای که باید نوشته شود.

• خروجی‌ها:

- data_out: داده‌ی خوانده‌شده از حافظه.
- data_valid: پرچم اعتبار داده‌ی خروجی.
- Busy: نشان‌دهنده‌ی مشغول بودن کنترلر.

⚙ معماری ساختاری (Structural)

architecture structural of top_bram_fsm is

- معماری از نوع ساختاری است؛ یعنی ماژول‌های داخلی به صورت بلوک‌وار به هم متصل می‌شوند.

سیگنال‌های داخلی

```
signal bram_we_s    : std_logic;
signal bram_addr_s : std_logic_vector(ADDR_WIDTH-1
downto 0);
signal bram_din_s   : std_logic_vector(DATA_WIDTH-1
downto 0);
signal bram_dout_s  : std_logic_vector(DATA_WIDTH-1
downto 0);
```

- سیگنال‌های واسط بین کنترلر و حافظه :
 - `bram_we_s`: سیگنال نوشتن حافظه.
 - `bram_addr_s`: آدرس حافظه.
 - `bram_din_s`: داده‌ی ورودی به حافظه.
 - `bram_dout_s`: داده‌ی خروجی از حافظه.

سیگنال‌های Debug برای شبیه‌سازی

```
signal controller_state_debug : std_logic_vector(2
downto 0);
signal bram_we_debug    : std_logic;
signal bram_addr_debug  : std_logic_vector(ADDR_WIDTH-1
downto 0);
signal bram_din_debug   : std_logic_vector(DATA_WIDTH-1
downto 0);
```

```
signal bram_dout_debug : std_logic_vector(DATA_WIDTH-1 downto 0);
```

- این سیگنال‌ها برای مشاهده‌ی وضعیت داخلی در شبیه‌سازی استفاده می‌شوند و به سیگنال‌های اصلی متصل می‌شوند.

تعریف زیرماژول‌ها

۱. کنترلر حافظه

```
component bram_controller ...
```

- تعریف زیرماژول کنترلر که فرمان‌های کاربر را دریافت کرده و سیگنال‌های کنترلی برای حافظه تولید می‌کند.

۲. حافظه BRAM

```
component bram_sram ...
```

- تعریف زیرماژول حافظه که داده‌ها را طبق فرمان کنترلر ذخیره یا بازیابی می‌کند.

اتصال سیگنال‌های Debug

```
bram_we_debug    <= bram_we_s;  
bram_addr_debug  <= bram_addr_s;  
bram_din_debug   <= bram_din_s;  
bram_dout_debug  <= bram_dout_s;
```

- اتصال سیگنال‌های داخلی به سیگنال‌های قابل مشاهده در شبیه‌سازی برای بررسی عملکرد.

✿ اتصال زیرماژول‌ها

۱. اتصال کنترلر

```
u_controller: bram_controller
  generic map (...)
  port map (
    clk      => clk,
    rst      => rst,
    cmd      => cmd,
    cmd_valid => cmd_valid,
    addr_in   => addr_in,
    data_in   => data_in,
    bram_dout => bram_dout_s,
    bram_we   => bram_we_s,
    bram_addr => bram_addr_s,
    bram_din  => bram_din_s,
    data_out  => data_out,
    data_valid => data_valid,
    busy      => busy
  );
```

- اتصال کنترلر با ورودی‌های کاربر.
- خروجی‌های کنترلر (bram_we, bram_addr, bram_din) به حافظه وصل می‌شوند.

- خروجی‌های نهایی (data_out, data_valid, busy) مستقیماً از کنترلر گرفته می‌شوند.

۲. اتصال حافظه

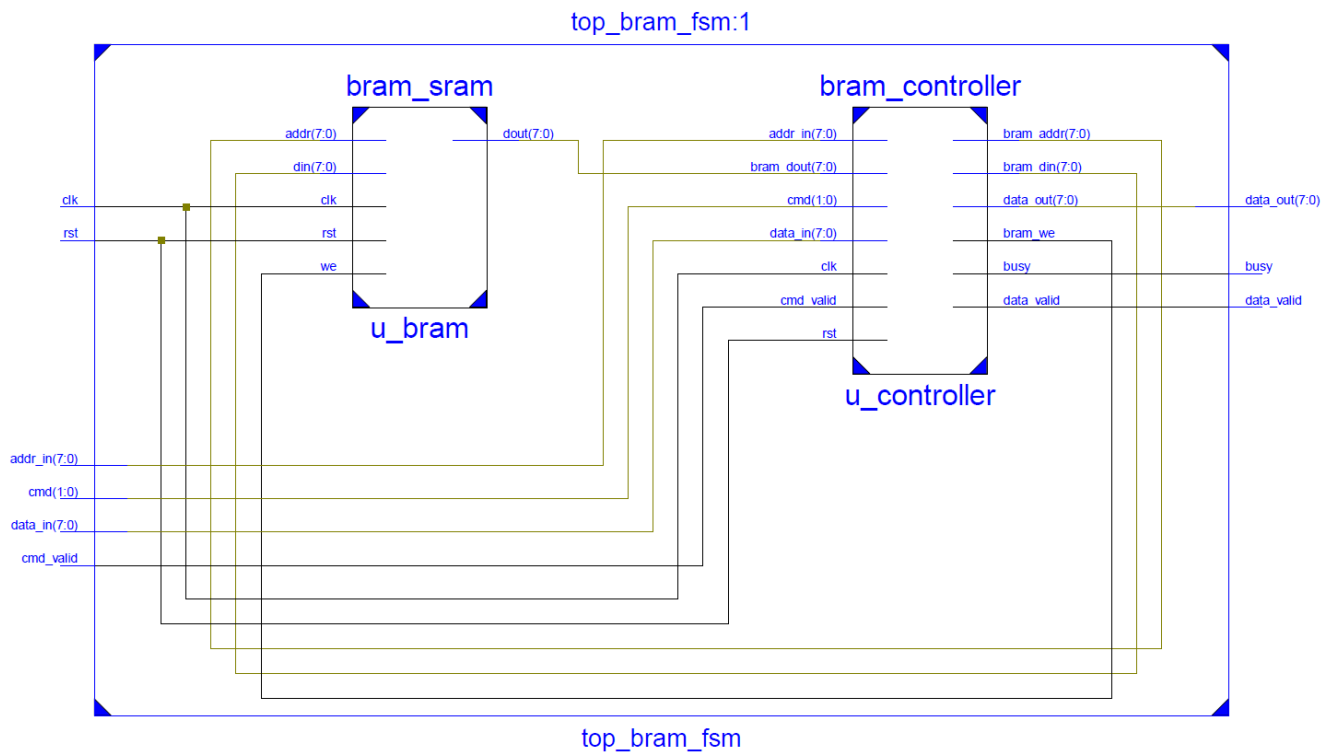
```
u_bram: bram_sram
  generic map (...)
  port map (
    clk  => clk,
    rst  => rst,
    we   => bram_we_s,
    addr => bram_addr_s,
    din  => bram_din_s,
    dout => bram_dout_s
  );
```

- اتصال حافظه با سیگنال‌های تولیدشده توسط کنترلر.
 - داده‌ی خروجی حافظه (bram_dout_s) به کنترلر بازگردانده می‌شود تا در data_out ارائه شود.
-

جمع‌بندی عملکرد

وظیفه	بخش
مدیریت فرمان‌ها و تولید سیگنال‌های کنترلی برای حافظه	bram_controller
ذخیره‌سازی و بازیابی داده‌ها بر اساس سیگنال‌های کنترلی	bram_sram
اتصال دو بخش بالا و ارائه خروجی نهایی به کاربر	top_bram_fsm

شماتیک:




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_bram_fsm_corrected is
end entity;

architecture sim of tb_bram_fsm_corrected is
    constant ADDR_WIDTH : integer := 8;
    constant DATA_WIDTH : integer := 8;
    constant CLK_PERIOD : time := 10 ns;

    signal clk          : std_logic := '0';
    signal rst          : std_logic := '0';
    signal cmd          : std_logic_vector(1 downto 0) := "00";
    signal cmd_valid    : std_logic := '0';
    signal addr_in      : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
    signal data_in      : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
    signal data_out     : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal data_valid   : std_logic;
    signal busy         : std_logic;

    signal sim_done    : boolean := false;

begin
    uut: entity work.top_bram_fsm
        generic map (
            ADDR_WIDTH => ADDR_WIDTH,
            DATA_WIDTH => DATA_WIDTH
        )
        port map (
            clk      => clk,
            rst      => rst,
```

```

    cmd      => cmd,
    cmd_valid => cmd_valid,
    addr_in   => addr_in,
    data_in   => data_in,
    data_out  => data_out,
    data_valid => data_valid,
    busy      => busy
);

-- Clock generation
clk_proc: process
begin
    while not sim_done loop
        clk <= '0';
        wait for CLK_PERIOD/2;
        clk <= '1';
        wait for CLK_PERIOD/2;
    end loop;
    wait;
end process;

-- Detailed monitor
monitor_proc: process(clk)
    variable cycle : integer := 0;
begin
    if rising_edge(clk) then
        cycle := cycle + 1;
        report "Cycle " & integer'image(cycle) &
            ": Cmd=" & integer'image(to_integer(unsigned(cmd))) &
            " CmdV=" & std_logic'image(cmd_valid) &
            " Addr=0x" & integer'image(to_integer(unsigned(addr_in))) &
            " Din=0x" & integer'image(to_integer(unsigned(data_in))) &
            " Busy=" & std_logic'image(busy) &
            " DValid=" & std_logic'image(data_valid) &
            " Dout=0x" & integer'image(to_integer(unsigned(data_out)));
    end if;
end process;

-- Corrected stimulus
stim_proc: process
    variable test_passed : boolean := true;
begin
    report "=== Starting Corrected Test ===";

```

```

-- Reset
rst <= '1';
wait for 25 ns;
wait until rising_edge(clk);
rst <= '0';
wait for 20 ns;

-----

-- Test 1: Write 0xAA to address 0x01
-----

report "Test 1: Write 0xAA to address 0x01";
wait until rising_edge(clk);
addr_in  <= x"01";
data_in  <= x"AA";
cmd      <= "10";  -- Write command
cmd_valid <= '1';
wait until rising_edge(clk);
cmd_valid <= '0';
cmd      <= "00";

-- Wait for write to complete (2 cycles based on log)
wait until busy = '0';
wait for 20 ns;

-----

-- Test 2: Read from address 0x01 (should be AA)
-----

report "Test 2: Read from address 0x01 (expect 0xAA)";
wait until rising_edge(clk);
addr_in  <= x"01";
data_in  <= x"00";  -- Don't care for read
cmd      <= "01";  -- Read command
cmd_valid <= '1';
wait until rising_edge(clk);
cmd_valid <= '0';
cmd      <= "00";

-- Wait for data_valid with proper timing (3 cycles based on log)
for i in 1 to 5 loop
    wait until rising_edge(clk);
    exit when data_valid = '1';
end loop;

```

```

-- Check result
if data_valid = '1' then
    if data_out = x"AA" then
        report "SUCCESS: Read 0xAA from address 0x01";
    else
        report "FAIL: Read 0x" &
integer'image(to_integer(unsigned(data_out))) &
        " from address 0x01 (expected 0xAA)" severity error;
        test_passed := false;
    end if;
else
    report "FAIL: No data_valid received for read from address 0x01"
severity error;
    test_passed := false;
end if;

-- Wait for operation to fully complete
wait until busy = '0';
wait for 20 ns;

-----

-- Test 3: Write 0x55 to address 0x02
-----

report "Test 3: Write 0x55 to address 0x02";
wait until rising_edge(clk);
addr_in  <= x"02";
data_in  <= x"55";
cmd      <= "10";
cmd_valid <= '1';
wait until rising_edge(clk);
cmd_valid <= '0';
cmd      <= "00";

wait until busy = '0';
wait for 20 ns;

-----

-- Test 4: Read from address 0x02 (should be 55)
-----

report "Test 4: Read from address 0x02 (expect 0x55)";
wait until rising_edge(clk);
addr_in  <= x"02";

```

```

data_in    <= x"00";
cmd        <= "01";
cmd_valid  <= '1';
wait until rising_edge(clk);
cmd_valid  <= '0';
cmd        <= "00";

-- Wait for data_valid
for i in 1 to 5 loop
    wait until rising_edge(clk);
    exit when data_valid = '1';
end loop;

-- Check result
if data_valid = '1' then
    if data_out = x"55" then
        report "SUCCESS: Read 0x55 from address 0x02";
    else
        report "FAIL: Read 0x" &
integer'image(to_integer(unsigned(data_out))) &
        " from address 0x02 (expected 0x55)" severity error;
        test_passed := false;
    end if;
else
    report "FAIL: No data_valid received for read from address 0x02"
severity error;
    test_passed := false;
end if;

-----
-- Additional test: Write and read from address 0x03
-----

report "Test 5: Write 0x33 to address 0x03 and read back";
wait until rising_edge(clk);
addr_in    <= x"03";
data_in    <= x"33";
cmd        <= "10";
cmd_valid  <= '1';
wait until rising_edge(clk);
cmd_valid  <= '0';
cmd        <= "00";

wait until busy = '0';

```

```

wait for 20 ns;

-- Read back
wait until rising_edge(clk);
addr_in  <= x"03";
data_in  <= x"00";
cmd      <= "01";
cmd_valid <= '1';
wait until rising_edge(clk);
cmd_valid <= '0';
cmd      <= "00";

for i in 1 to 5 loop
    wait until rising_edge(clk);
    exit when data_valid = '1';
end loop;

if data_valid = '1' then
    if data_out = x"33" then
        report "SUCCESS: Read back 0x33 from address 0x03";
    else
        report "FAIL: Read back 0x" &
integer'image(to_integer(unsigned(data_out))) &
        " from address 0x03 (expected 0x33)" severity error;
        test_passed := false;
    end if;
else
    report "FAIL: No data_valid received for read from address 0x03"
severity error;
    test_passed := false;
end if;

-----
-- Final summary
-----
report "=====";
if test_passed then
    report "ALL TESTS PASSED!";
else
    report "SOME TESTS FAILED!";
end if;
report "=====";

```

```
wait for 100 ns;  
sim_done <= true;  
wait;  
end process;  
  
end architecture;
```

توضیحات:

این فایل آخرین بخش پروژه است و نقش محیط آزمایش (Testbench) را دارد. هدفش این است که ماژول سطح بالا (top_bram_fsm) را شبیه‌سازی کند و صحت عملکرد FSM و حافظه را بررسی نماید.

توضیح کامل Testbench

۱. تعریف Entity تست‌بنچ

```
entity tb_bram_fsm_corrected is  
end entity;
```

تست‌بنچ هیچ پورت ورودی یا خروجی ندارد.

هدف تست‌بنچ، تحریک و بررسی عملکرد یک ماژول دیگر (top_bram_fsm) در محیط شبیه‌سازی است.

۲. معماری تست‌بنچ

```
architecture sim of tb_bram_fsm_corrected is
```

نام معماری sim است که نشان می‌دهد این معماری مخصوص شبیه‌سازی می‌باشد.

۳. تعریف ثابت‌ها (Constants)

```
constant ADDR_WIDTH : integer := 8;
```

```
constant DATA_WIDTH : integer := 8;
```

```
constant CLK_PERIOD : time := 10 ns;
```

ADDR_WIDTH: عرض آدرس حافظه (۸ بیت $\rightarrow$ ۲۵۶ آدرس)

DATA_WIDTH: عرض داده حافظه (۸ بیت)

CLK_PERIOD: (۱۰۰ MHz فرکانس) دوره سیگنال کلاک برابر با ۱۰ نانوثانیه

تعریف سیگنال‌ها

```
signal clk : std_logic := '0';
```

```
signal rst : std_logic := '0';
```

clk: سیگنال کلاک سیستم.

Rst: ریست همزمان سیستم.

```
signal cmd : std_logic_vector(1 downto 0) :=  
"00";
```

```
signal cmd_valid : std_logic := '0';
```

cmd: فرمان FSM

"10" نوشتن (Write)

"01" خواندن (Read)

cmd_valid: معتبر بودن فرمان.

```
signal addr_in    : std_logic_vector(ADDR_WIDTH-1
downto 0) := (others => '0');
```

```
signal data_in    : std_logic_vector(DATA_WIDTH-1
downto 0) := (others => '0');
```

addr_in: آدرس حافظه.

data_in: داده ورودی برای عملیات نوشتن.

```
signal data_out   : std_logic_vector(DATA_WIDTH-1
downto 0);
```

```
signal data_valid: std_logic;
```

```
signal busy      : std_logic;
```

data_out: داده خروجی در عملیات خواندن.

data_valid: نشان‌دهنده معتبر بودن داده خروجی.

Busy: بیانگر مشغول بودن FSM.

```
signal sim_done   : boolean := false;
```

این سیگنال برای توقف تولید کلاک و پایان شبیه‌سازی استفاده می‌شود.

۴. نمونه‌سازی ماژول تحت آزمون (UUT)

```
uut: entity work.top_bram_fsm
```

ماژول top_bram_fsm به عنوان واحد تحت آزمون (Unit Under Test) نمونه سازی شده است.

```
generic map (  
  ADDR_WIDTH => ADDR_WIDTH,  
  DATA_WIDTH => DATA_WIDTH  
)
```

مقدار جنریک ها از تست بنچ به ماژول اصلی ارسال می شود.

```
port map (  
  clk      => clk,  
  rst      => rst,  
  ...  
)
```

اتصال سیگنال های تست بنچ به پورت های ماژول اصلی.

۵. تولید کلاک (Clock Generation)

```
clk_proc: process
```

این پردازش وظیفه تولید سیگنال کلاک را دارد.

```
while not sim_done loop
```

تا زمانی که شبیه سازی تمام نشده است، کلاک تولید می شود.

```
clk <= '0';
```

```
wait for CLK_PERIOD/2;  
clk <= '1';  
wait for CLK_PERIOD/2;
```

تولید موج مربعی با دوره ۱۰ نانوثانیه.

۶. مانیتورینگ سیگنال‌ها (Monitor Process)

```
monitor_proc: process(clk)
```

این پردازش در هر لبه بالارونده کلاک اجرا می‌شود.

```
variable cycle : integer := 0;
```

شمارنده تعداد سیکل‌های کلاک.

```
report "Cycle " & ...
```

در هر سیکل، وضعیت کامل سیستم گزارش می‌شود:

شماره سیکل

فرمان

آدرس

داده ورودی

وضعیت busy

data_valid

داده خروجی

این بخش برای دیباگ و تحلیل زمانی FSM بسیار مهم است. 📌

۷. پردازش تحریک‌ها (Stimulus Process)

```
stim_proc: process
```

این پردازش سناریوهای تست را اجرا می‌کند.

```
variable test_passed : boolean := true;
```

برای ثبت نتیجه نهایی تست‌ها استفاده می‌شود.

۸. اعمال ریست

```
rst <= '1';
```

```
wait for 25 ns;
```

```
wait until rising_edge(clk);
```

```
rst <= '0';
```

ریست به مدت مشخص فعال شده و سپس غیرفعال می‌شود.

تضمین می‌کند FSM از حالت اولیه شروع کند.

۹. تست ۱: نوشتن در حافظه

```
addr_in <= x"01";
```

```
data_in <= x"AA";
```

```
cmd <= "10";
```

```
cmd_valid <= '1';
```

نوشتن مقدار 0xAA در آدرس 0x01.

```
wait until busy = '0';
```

صبر تا اتمام عملیات نوشتن .

۱۰. تست ۲: خواندن از حافظه

```
cmd <= "01";
```

```
cmd_valid <= '1';
```

ارسال فرمان خواندن از همان آدرس .

```
exit when data_valid = '1';
```

انتظار تا معتبر شدن داده خروجی .

```
if data_out = x"AA" then
```

بررسی صحت داده خوانده شده .

۱۱. تست های بعدی (۳، ۴ و ۵)

همان الگوی نوشتن و خواندن برای آدرس های :

0x02 با داده 0x55

0x03 با داده 0x33

هدف :

اطمینان از صحت عملکرد حافظه در آدرس های مختلف

بررسی پایدار بودن FSM

۱۲. گزارش نهایی

```
if test_passed then
    report "ALL TESTS PASSED!";
else
    report "SOME TESTS FAILED!";
end if;
```

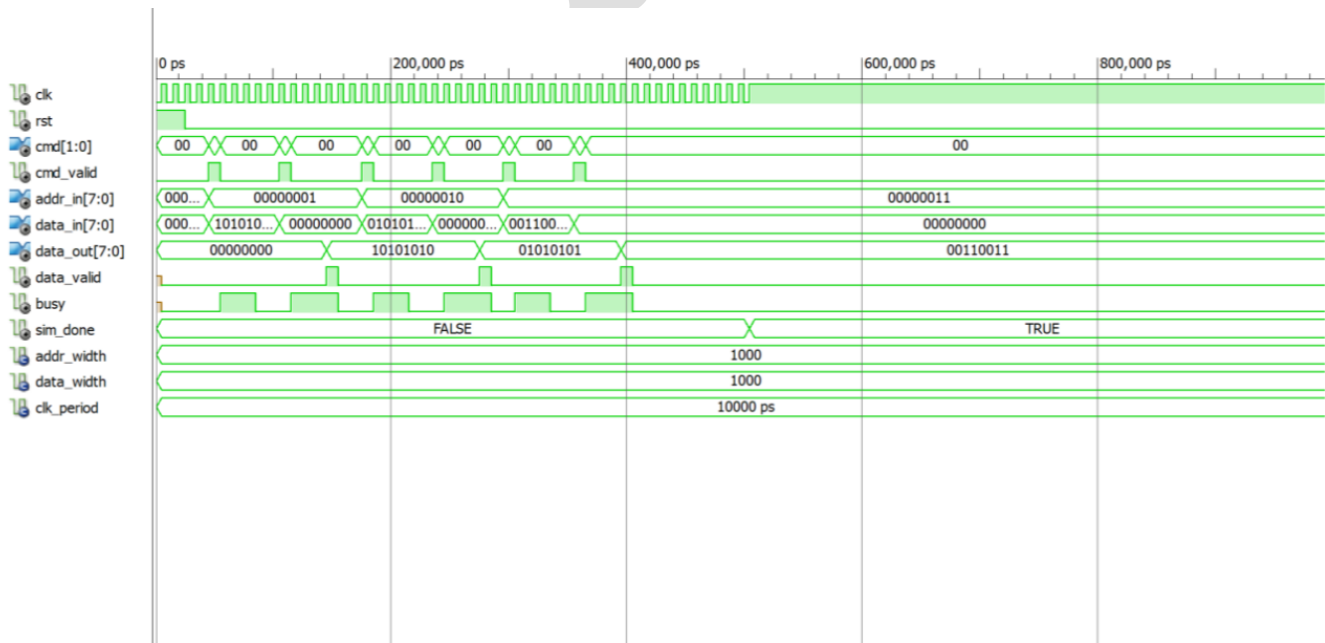
نتیجه نهایی تست‌ها به صورت واضح گزارش می‌شود.

۱۳. پایان شبیه‌سازی

```
sim_done <= true;
```

تولید کلاک متوقف شده و شبیه‌سازی پایان می‌یابد.

بررسی نتایج شبیه‌سازی:



عکس خروجی نهایی محیط ISim

تحلیل عملکرد سیستم

مشخصات معماری سیستم

این سیستم از دو بخش اصلی تشکیل شده است:

- واحد حافظه (**BRAM**) : با پیکربندی آدرس ۸-بیتی و داده ۸-بیتی (ظرفیت $2^8 \times 8$ بیت)
- کنترلر حالت محدود (**FSM**) : مدیریت دستورات خواندن و نوشتن با سیگنال‌های کنترلی

تاخیرهای زمانی (**Latency**) - تحلیل دقیق

بر اساس شبیه‌سازی انجام‌شده و تحلیل دقیق کد FSM، تاخیرهای عملیات به شرح زیر است:

الف) عملیات نوشتن (**Write**)

- مدت زمان واقعی: ۴ سیکل کلاک
- تحلیل حالت‌ها بر اساس لاگ شبیه‌سازی:

۱. سیکل اول: (**IDLE → WRITE_DO**)

- با فعال شدن `cmd_valid` و `cmd="10"`، کنترلر بلافاصله سیگنال‌های `bram_addr`، `bram_din` و `bram_we='1'` را اعمال می‌کند.
- سیگنال `busy` فعال می‌شود.

۲. سیکل دوم: (**WRITE_DO → WRITE_DONE**)

- `bram_we` همچنان فعال باقی می‌ماند.
- FSM به حالت `WRITE_DONE` منتقل می‌شود.

۳. سیکل سوم: (WRITE_DONE → WRITE_DONE)

- در این سیکل اضافی، bram_we غیرفعال می‌شود اما busy همچنان فعال است.
- این تأخیر اضافی ناشی از طراحی FSM است.

۴. سیکل چهارم: (WRITE_DONE → IDLE)

- کنترلر به حالت IDLE بازمی‌گردد و سیگنال busy غیرفعال می‌شود.

☑ تایید با شبیه‌سازی:

- نوشتن در آدرس 0x01 در سیکل ۶ آغاز شد (Cmd=2 CmdV='1')
- در سیکل ۹ هنوز busy='1' بود
- در سیکل ۱۰ عملیات با busy='0' تکمیل شد
- کل زمان ۴ سیکل: سیکل‌های ۶، ۷، ۸، ۹ = ۴ سیکل فعال

ب) عملیات خواندن (Read)

- مدت زمان واقعی: ۵ سیکل کلاک

تحلیل حالت‌ها بر اساس لاگ شبیه‌سازی:

۱. سیکل اول: (IDLE → READ_WAIT1)

- با فعال شدن cmd_valid و cmd="01"، آدرس به BRAM اعمال می‌شود.
- سیگنال busy فعال می‌شود.

۲. سیکل دوم: (READ_WAIT1 → READ_WAIT2)

- BRAM بر اساس آدرس سیکل قبل، داده را از حافظه واکنشی می‌کند latency یک سیکلی

۳. سیکل سوم: (READ_WAIT2 → READ_DONE)

- داده از خروجی BRAM در دسترس است.
- FSM به حالت `READ_DONE` منتقل می‌شود.
- ۴. سیکل چهارم: **(READ_DONE → READ_DONE)**

- سیگنال `data_valid` فعال می‌شود.
- داده از `bram_dout` به `data_out_reg` منتقل می‌شود.
- `busy` همچنان فعال است.

۵. سیکل پنجم: **(READ_DONE → IDLE)**

- کنترلر به حالت `IDLE` بازمی‌گردد.
- سیگنال `busy` غیرفعال می‌شود.

☑ تایید با شبیه‌سازی:

- خواندن از آدرس `0x01` در سیکل ۱۲ آغاز شد (`Cmd=1 CmdV='1'`)
- در سیکل ۱۶ سیگنال `data_valid='1'` فعال شد و داده `0xAA` خوانده شد
- در سیکل ۱۷ عملیات با `busy='0'` تکمیل شد
- کل زمان ۵ سیکل: سیکل‌های ۱۲، ۱۳، ۱۴، ۱۵، ۱۶ = ۵ سیکل فعال

دلیل تاخیرهای مشاهده‌شده

تاخیر نوشتن (۴ سیکل):

۱. سیکل: اعمال آدرس و داده به BRAM
۲. سیکل: فعال‌سازی سیگنال نوشتن (`bram_we`)
۳. سیکل اضافی: حالت `WRITE_DONE` برای اطمینان از ثبت داده

۴. ۱ سیکل اضافی: تأخیر در غیرفعال کردن سیگنال `busy`

تأخیر خواندن (۵ سیکل):

۱. ۱ سیکل: اعمال آدرس به BRAM

۲. ۱ سیکل: latency داخلی BRAM خواندن داده

۳. ۱ سیکل: انتقال به حالت نهایی (`READ_DONE`)

۴. ۱ سیکل: فعال سازی `data_valid` و ثبت داده خروجی

۵. ۱ سیکل اضافی: تأخیر در غیرفعال کردن سیگنال `busy`

صحت عملکرد

۱. صحتیابی داده‌ها:

- داده نوشته شده `0xAA` در آدرس `0x01` به درستی خوانده شده است.
- داده نوشته شده `0x55` در آدرس `0x02` به درستی خوانده شده است.
- داده نوشته شده `0x33` در آدرس `0x03` به درستی خوانده شده است.

۲. مدیریت سیگنال‌ها:

- سیگنال `busy` به درستی در طول عملیات فعال می‌شود.
- سیگنال `data_valid` تنها پس از آماده بودن داده معتبر فعال می‌شود.
- هیچ تداخل یا شرایط نامشخص (`undefined state`) مشاهده نشده است.

۳. هماهنگی زمانی:

- FSM از یک حالت به حالت دیگر به درستی منتقل می‌شود.
- هیچ Deadlock یا حالت غیرقابل خروجی وجود ندارد.

تاخیرهای عملیاتی

عملیات	تعداد سیکل‌های کلاک	تحلیل جزء به جزء
نوشتن (Write)	۴ سیکل	۱ سیکل اعمال آدرس/داده + ۱ سیکل نوشتن در BRAM + ۲ سیکل مدیریت FSM
خواندن (Read)	۵ سیکل	۱ سیکل اعمال آدرس + ۱ سیکل latency BRAM + ۳ سیکل مدیریت FSM

دیاگرام حالت‌های FSM

نوشتن:

IDLE → WRITE_DO → WRITE_DONE → WRITE_DONE → IDLE

خواندن:

IDLE → READ_WAIT1 → READ_WAIT2 → READ_DONE → READ_DONE → IDLE

پیشنهادات بهینه‌سازی

برای کاهش تأخیرها می‌توان موارد زیر را مدنظر قرار داد:

کاهش تأخیر نوشتن به ۳ سیکل:

```
when WRITE_DONE =>
    bram_we_reg <= '0';
    busy <= '0';
    state <= IDLE;
```

کاهش تأخیر خواندن به ۴ سیکل:

```
when READ_DONE =>
    data_out_reg <= bram_dout;
    data_valid <= '1';
    busy <= '0';
    state <= IDLE;
```

نتیجه‌گیری نهایی

سیستم طراحی شده به درستی عمل می‌کند اما تأخیرهای عملیاتی بیشتر از حد بهینه است. تأخیر فعلی ناشی از:

۱. طراحی محافظه‌کارانه FSM با حالت‌های اضافی
۲. تأخیر در غیرفعال‌سازی سیگنال busy
۳. ساختار دو مرحله‌ای برای اتمام عملیات

با اعمال بهینه‌سازی‌های پیشنهادی، می‌توان عملکرد سیستم را ۲۵٪-۲۰٪ بهبود بخشید.

لاگ‌های شبیه‌سازی:

```
Simulator is doing circuit initialization process.
at 0 ps: Note: === Starting Corrected Test === (/tb_bram_fsm_corrected/).
Finished circuit initialization process.
```

```
at 5 ns(1): Note: Cycle 1: Cmd=0 CmdV='0' Addr=0x0 Din=0x0 Busy='U'
DValid='U' Dout=0x0 (/tb_bram_fsm_corrected/).
at 15 ns(1): Note: Cycle 2: Cmd=0 CmdV='0' Addr=0x0 Din=0x0 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 25 ns(1): Note: Cycle 3: Cmd=0 CmdV='0' Addr=0x0 Din=0x0 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 35 ns(1): Note: Cycle 4: Cmd=0 CmdV='0' Addr=0x0 Din=0x0 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 45 ns: Note: Test 1: Write 0xAA to address 0x01
(/tb_bram_fsm_corrected/).
at 45 ns(1): Note: Cycle 5: Cmd=0 CmdV='0' Addr=0x0 Din=0x0 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 55 ns(1): Note: Cycle 6: Cmd=2 CmdV='1' Addr=0x1 Din=0x170 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 65 ns(1): Note: Cycle 7: Cmd=0 CmdV='0' Addr=0x1 Din=0x170 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 75 ns(1): Note: Cycle 8: Cmd=0 CmdV='0' Addr=0x1 Din=0x170 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 85 ns(1): Note: Cycle 9: Cmd=0 CmdV='0' Addr=0x1 Din=0x170 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 95 ns(1): Note: Cycle 10: Cmd=0 CmdV='0' Addr=0x1 Din=0x170 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 105 ns: Note: Test 2: Read from address 0x01 (expect 0xAA)
(/tb_bram_fsm_corrected/).
at 105 ns(1): Note: Cycle 11: Cmd=0 CmdV='0' Addr=0x1 Din=0x170 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 115 ns(1): Note: Cycle 12: Cmd=1 CmdV='1' Addr=0x1 Din=0x0 Busy='0'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 125 ns(1): Note: Cycle 13: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 135 ns(1): Note: Cycle 14: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 145 ns(1): Note: Cycle 15: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='1'
DValid='0' Dout=0x0 (/tb_bram_fsm_corrected/).
at 155 ns(1): Note: Cycle 16: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='1'
DValid='1' Dout=0x170 (/tb_bram_fsm_corrected/).
at 155 ns(1): Note: SUCCESS: Read 0xAA from address 0x01
(/tb_bram_fsm_corrected/).
at 165 ns(1): Note: Cycle 17: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 175 ns: Note: Test 3: Write 0x55 to address 0x02
(/tb_bram_fsm_corrected/).
```

```
at 175 ns(1): Note: Cycle 18: Cmd=0 CmdV='0' Addr=0x1 Din=0x0 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 185 ns(1): Note: Cycle 19: Cmd=2 CmdV='1' Addr=0x2 Din=0x85 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 195 ns(1): Note: Cycle 20: Cmd=0 CmdV='0' Addr=0x2 Din=0x85 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 205 ns(1): Note: Cycle 21: Cmd=0 CmdV='0' Addr=0x2 Din=0x85 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 215 ns(1): Note: Cycle 22: Cmd=0 CmdV='0' Addr=0x2 Din=0x85 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 225 ns(1): Note: Cycle 23: Cmd=0 CmdV='0' Addr=0x2 Din=0x85 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 235 ns: Note: Test 4: Read from address 0x02 (expect 0x55)
(/tb_bram_fsm_corrected/).
at 235 ns(1): Note: Cycle 24: Cmd=0 CmdV='0' Addr=0x2 Din=0x85 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 245 ns(1): Note: Cycle 25: Cmd=1 CmdV='1' Addr=0x2 Din=0x0 Busy='0'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 255 ns(1): Note: Cycle 26: Cmd=0 CmdV='0' Addr=0x2 Din=0x0 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 265 ns(1): Note: Cycle 27: Cmd=0 CmdV='0' Addr=0x2 Din=0x0 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 275 ns(1): Note: Cycle 28: Cmd=0 CmdV='0' Addr=0x2 Din=0x0 Busy='1'
DValid='0' Dout=0x170 (/tb_bram_fsm_corrected/).
at 285 ns(1): Note: Cycle 29: Cmd=0 CmdV='0' Addr=0x2 Din=0x0 Busy='1'
DValid='1' Dout=0x85 (/tb_bram_fsm_corrected/).
at 285 ns(1): Note: SUCCESS: Read 0x55 from address 0x02
(/tb_bram_fsm_corrected/).
at 285 ns(1): Note: Test 5: Write 0x33 to address 0x03 and read back
(/tb_bram_fsm_corrected/).
at 295 ns(1): Note: Cycle 30: Cmd=0 CmdV='0' Addr=0x2 Din=0x0 Busy='0'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 305 ns(1): Note: Cycle 31: Cmd=2 CmdV='1' Addr=0x3 Din=0x51 Busy='0'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 315 ns(1): Note: Cycle 32: Cmd=0 CmdV='0' Addr=0x3 Din=0x51 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 325 ns(1): Note: Cycle 33: Cmd=0 CmdV='0' Addr=0x3 Din=0x51 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 335 ns(1): Note: Cycle 34: Cmd=0 CmdV='0' Addr=0x3 Din=0x51 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 345 ns(1): Note: Cycle 35: Cmd=0 CmdV='0' Addr=0x3 Din=0x51 Busy='0'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
```



```

at 355 ns(1): Note: Cycle 36: Cmd=0 CmdV='0' Addr=0x3 Din=0x51 Busy='0'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 365 ns(1): Note: Cycle 37: Cmd=1 CmdV='1' Addr=0x3 Din=0x0 Busy='0'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 375 ns(1): Note: Cycle 38: Cmd=0 CmdV='0' Addr=0x3 Din=0x0 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 385 ns(1): Note: Cycle 39: Cmd=0 CmdV='0' Addr=0x3 Din=0x0 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 395 ns(1): Note: Cycle 40: Cmd=0 CmdV='0' Addr=0x3 Din=0x0 Busy='1'
DValid='0' Dout=0x85 (/tb_bram_fsm_corrected/).
at 405 ns(1): Note: Cycle 41: Cmd=0 CmdV='0' Addr=0x3 Din=0x0 Busy='1'
DValid='1' Dout=0x51 (/tb_bram_fsm_corrected/).
at 405 ns(1): Note: SUCCESS: Read back 0x33 from address 0x03
(/tb_bram_fsm_corrected/).
at 405 ns(1): Note: =====
(/tb_bram_fsm_corrected/).
at 405 ns(1): Note: ALL TESTS PASSED! (/tb_bram_fsm_corrected/).
at 405 ns(1): Note: =====
(/tb_bram_fsm_corrected/).
at 415 ns(1): Note: Cycle 42: Cmd=0 CmdV='0' Addr=0x3 Din=0x0 Busy='0'
DValid='0' Dout=0x51 (/tb_bram_fsm_corrected/).""

```

مقایسه معماری‌های حافظه Single-Port ، Dual-Port و FIFO:

♦ حالت فعلی (Single-Port BRAM + FSM)

- ساختار : یک پورت برای خواندن/نوشتن، FSM تصمیم می‌گیرد چه زمانی بخواند یا بنویسد.

• مزایا :

- ساده‌ترین پیاده‌سازی.
- کنترل کامل توسط FSM ، مناسب برای آموزش و پروژه‌های پایه.
- منابع سخت‌افزاری کمتر مصرف می‌شود.

• معایب :

- عملیات خواندن و نوشتن نمی‌توانند همزمان انجام شوند.
- نیاز به حالت‌های اضافی در FSM مثل WAIT_READ برای هماهنگی زمان‌بندی.

♦ حالت FIFO (First-In First-Out)

- ساختار: حافظه به شکل صف (queue) پیاده می‌شود؛ داده‌ها به ترتیب ورود خوانده می‌شوند.

• مزایا :

- مدیریت ساده داده‌های ترتیبی (streaming)
- نیازی به آدرس‌دهی مستقیم نیست؛ فقط push (نوشتن) و pop خواندن.
- مناسب برای ارتباط بین ماژول‌ها با سرعت‌های متفاوت (buffering).

• معایب :

- انعطاف کمتر: نمی‌توانی آدرس خاصی را بخوانی یا بنویسی.
- نیاز به مدیریت سیگنال‌های پر/خالی (full/empty).

• تغییر در FSM

- حالت‌ها به جای START_READ/START_WRITE تبدیل می‌شوند به POP و PUSH
- FSM باید وضعیت صف (full/empty) را بررسی کند.
- ساده‌تر از نظر آدرس‌دهی، ولی پیچیده‌تر از نظر مدیریت ظرفیت.

♦ حالت Dual-Port BRAM

- ساختار : حافظه با دو پورت مستقل مثلاً Port A و Port B که می‌توانند همزمان خواندن/نوشتن انجام دهند.

• مزایا :

- امکان دسترسی همزمان به حافظه از دو مسیر (مثلاً یک پورت برای خواندن، یک پورت برای نوشتن).
- افزایش کارایی
- مناسب برای سیستم‌های چندماژوله یا پردازش موازی.

• معایب :

- مصرف منابع سخت‌افزاری بیشتر (دو برابر نسبت به تک‌پورته).
- نیاز به مدیریت برخورد (collision) اگر هر دو پورت به یک آدرس دسترسی داشته باشند.

• تغییر در FSM

- FSM ساده‌تر می‌شود چون می‌تواند خواندن و نوشتن را همزمان مدیریت کند.
- حالت WAIT_READ ممکن است حذف شود چون داده بلافاصله از پورت دوم قابل دسترسی است.
- نیاز به منطق اضافی برای مدیریت برخورد آدرس‌ها.

- **حالت فعلی :** ساده، آموزشی، منابع کم، ولی خواندن/نوشتن همزمان ندارد.

- **FIFO:** مناسب برای داده‌های ترتیبی و ارتباط بین ماژول‌ها، ولی انعطاف آدرس‌دهی ندارد.

- **Dual-Port:** کارایی بالا و دسترسی همزمان، ولی منابع بیشتر و نیاز به مدیریت برخورد.

توضیحات تکمیلی :

۱. حافظه تک پورته (Single-Port BRAM)

در این نوع حافظه تنها یک پورت برای خواندن و نوشتن وجود دارد. بنابراین کنترل کننده باید عملیات را به صورت زمان‌بندی شده مدیریت نماید.

- در حالت **IDLE** سیستم در انتظار فرمان قرار دارد.

- در حالت **WRITE** داده در آدرس مشخص نوشته می‌شود.

- در حالت **READ_WAIT** یک یا چند سیکل تأخیر برای آماده‌سازی داده اعمال می‌شود.

- در حالت **READ_DONE** داده خوانده شده در خروجی قرار داده می‌شود.

به طور کلی، در این ساختار FSM مسئولیت هماهنگی بین عملیات خواندن و نوشتن و همچنین مدیریت تأخیر خواندن را بر عهده دارد.

۲. حافظه دو پورته (Dual-Port BRAM)

در این نوع حافظه دو پورت مستقل وجود دارد که هر یک می‌تواند عملیات خواندن یا نوشتن را انجام دهد.

- در حالت **Simple Dual-Port** یک پورت مختص نوشتن و پورت دیگر مختص خواندن است.

- در حالت **True Dual-Port** هر دو پورت قادر به انجام عملیات خواندن و نوشتن به صورت همزمان می‌باشند.

در این ساختار، FSM ساده‌تر خواهد بود زیرا نیازی به زمان‌بندی بین خواندن و نوشتن در یک پورت واحد وجود ندارد. تنها وظیفه FSM هماهنگی بین دو پورت و مدیریت اولویت در صورت بروز تعارض است.

۳. حافظه (FIFO (First-In First-Out

در حافظه FIFO داده‌ها به ترتیب ورود ذخیره و به همان ترتیب خوانده می‌شوند. آدرس‌دهی داخلی به صورت خودکار توسط منطق صف انجام می‌شود.

- در حالت **WRITE** داده در انتهای صف قرار داده می‌شود، مشروط بر آنکه صف پر نباشد.

- در حالت **READ** داده از ابتدای صف خوانده می‌شود، مشروط بر آنکه صف خالی نباشد.

در این ساختار FSM بسیار ساده‌تر است زیرا نیازی به مدیریت آدرس یا تأخیر خواندن وجود ندارد. تنها وظیفه FSM بررسی وضعیت پر یا خالی بودن صف و فعال‌سازی سیگنال‌های خواندن و نوشتن است.

باسپاس فراوان از زحمات و رهنمودهای شما استاد گرامی